

1. Understanding the Need for Styling React Components

Why Styling is Important

- Makes the application visually appealing and user-friendly
- Improves user experience by providing clear visual hierarchy
- Helps in branding and maintaining consistency across the application
- Enhances readability and accessibility
- Provides visual feedback for user interactions

Challenges in Styling React Applications

- CSS rules are global by default, which can cause conflicts
- Large applications can have CSS class name collisions
- Maintaining styles across multiple components can become difficult
- Dynamic styling based on state or props requires special handling
- Performance considerations when styles change frequently

Different Ways to Style React Components

Method	Description
Regular CSS	Traditional CSS files imported into components
CSS Modules	Locally scoped CSS classes
Inline Styles	Styles applied directly to elements using style attribute
Styled Components	CSS-in-JS library using tagged template literals

Sass/SCSS	CSS with variables, nesting, and mixins
Tailwind CSS	Utility-first CSS framework
Material-UI	Component library with built-in styling

2. CSS Modules

What are CSS Modules?

- CSS Modules are CSS files where all class names are locally scoped by default
- Each component gets its own unique CSS class names
- Prevents class name collisions across components
- The class names are generated automatically during build process

Why Use CSS Modules

- No global namespace pollution
- No class name conflicts between components
- Explicit dependencies: you import only what you need
- Component-based styling: styles are tied to specific components
- Easy to maintain and refactor

How CSS Modules Work

1. Write CSS in a file with `.module.css` extension
2. Import the file as an object in JavaScript
3. Use the imported object to reference class names
4. During build, class names are hashed to be unique

CSS Modules Example

File: Button.module.css

css

```
.button {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.button:hover {
  background-color: darkblue;
}

.primary {
  background-color: green;
}

.secondary {
  background-color: gray;
}
```

File: Button.js

jsx

```
import React from 'react';
import styles from './Button.module.css';

function Button({ variant, children }) {
  const className = `${styles.button} ${variant === 'primary' ?
styles.primary : ''}`;

  return (
    <button className={className}>
      {children}
    </button>
  );
}
```

```
export default Button;
```

Importing CSS Modules

```
jsx
```

```
//  Correct - Import as an object  
import styles from './Component.module.css';
```

```
// Usage
```

```
<div className={styles.container}>...</div>
```

Using Multiple Classes with CSS Modules

```
jsx
```

```
// Method 1: Template literals
```

```
<div className={` ${styles.container} ${styles.active} `}>...</div>
```

```
// Method 2: Array join
```

```
<div className={[styles.container, styles.active].join(' ')}>...</div>
```

```
// Method 3: Using classnames library
```

```
import classNames from 'classnames';
```

```
<div className={classNames(styles.container, styles.active)}>...</div>
```

Composing Classes in CSS Modules

```
css
```

```
/* Button.module.css */  
.base {  
  padding: 10px;  
  border-radius: 5px;  
}
```

```
.primary {  
  composes: base;  
  background-color: blue;  
  color: white;  
}  
  
.secondary {  
  composes: base;  
  background-color: gray;  
  color: black;  
}
```

3. Inline Styles

What are Inline Styles?

- Styles applied directly to elements using the `style` attribute
- Styles are defined as JavaScript objects
- CamelCase property names (like `backgroundColor` instead of `background-color`)
- Styles are applied directly to the element's DOM

Why Use Inline Styles

- No external CSS files needed
- Dynamic styling based on state or props
- No class name conflicts
- Easy to implement conditional styling
- Styles are scoped to the specific element

Inline Styles Example

File: StyledComponent.js

jsx

```
import React, { useState } from 'react';

function StyledComponent() {
  const [isActive, setIsActive] = useState(false);

  const styles = {
    container: {
      padding: '20px',
      backgroundColor: isActive ? 'blue' : 'gray',
      borderRadius: '10px',
      transition: 'all 0.3s ease'
    },
    heading: {
      color: 'white',
      fontSize: '24px',
      fontFamily: 'Arial, sans-serif'
    },
    button: {
      padding: '10px 20px',
      backgroundColor: 'white',
      color: 'black',
      border: 'none',
      borderRadius: '5px',
      cursor: 'pointer'
    }
  };

  return (
    <div style={styles.container}>
      <h1 style={styles.heading}>Hello World</h1>
      <button
        style={styles.button}
        onClick={() => setIsActive(!isActive)}
      >
        Toggle Active
      </button>
    </div>
  );
}
```

```
export default StyledComponent;
```

Dynamic Inline Styles Based on Props

```
jsx
```

```
function Message({ type, message }) {  
  const styles = {  
    padding: '15px',  
    borderRadius: '5px',  
    backgroundColor: type === 'success' ? 'green' :  
                     type === 'error' ? 'red' :  
                     type === 'warning' ? 'yellow' : 'gray',  
    color: type === 'warning' ? 'black' : 'white'  
  };  
  
  return <div style={styles}>{message}</div>;  
  
}
```

Conditional Inline Styles

```
jsx
```

```
function ToggleButton({ isOn }) {  
  const buttonStyle = {  
    padding: '15px 30px',  
    backgroundColor: isOn ? 'green' : 'red',  
    color: 'white',  
    border: 'none',  
    borderRadius: '5px',  
    cursor: 'pointer',  
    transition: 'background-color 0.3s ease'  
  };  
  
  return (  
    <button style={buttonStyle}>  
      {isOn ? 'ON' : 'OFF'}  
    </button>  
  );  
}
```

```
}
```

Complex Inline Styles with Calculations

```
jsx
```

```
function ProgressBar({ progress }) {
  const styles = {
    container: {
      width: '100%',
      height: '20px',
      backgroundColor: '#e0e0e0',
      borderRadius: '10px',
      overflow: 'hidden'
    },
    fill: {
      width: `${Math.min(progress, 100)}%`,
      height: '100%',
      backgroundColor: progress > 70 ? 'green' :
        progress > 40 ? 'orange' : 'red',
      transition: 'width 0.5s ease'
    },
    label: {
      textAlign: 'center',
      marginTop: '5px',
      fontSize: '14px'
    }
  };

  return (
    <div>
      <div style={styles.container}>
        <div style={styles.fill}></div>
      </div>
      <div style={styles.label}>{progress}% Complete</div>
    </div>
  );
}
```

4. Comparison Table

Feature	CSS Modules	Inline Styles
Scope	Component-scoped	Element-scoped
CSS Features	All CSS features	Limited to JavaScript styles
Performance	Fast (compiled to CSS)	Good (but recalculated on renders)
Media Queries	Supported	Not supported
Pseudo-classes	Supported (:hover, :focus)	Not supported (need JavaScript)
Animations	Supported	Not supported
Debugging	Hashed class names	Easier to debug
Reusability	Easy (composable)	Harder (object duplication)
Dynamic Styling	Requires class switching	Native (use state/props)
Learning Curve	Low	Low

5. Best Practices

CSS Modules Best Practices

- Use meaningful class names (e.g., `.header`, `.button`)

- Keep components and their styles in the same folder
- Use composition for reusable styles
- Avoid deep nesting (keep specificity low)
- Use BEM naming convention within modules

Inline Styles Best Practices

- Only use for dynamic styles that depend on state
- Extract style objects outside component when possible
- Use for prototyping or small components
- Avoid complex style logic inside render
- Consider performance when styles update frequently

General Styling Best Practices

- Choose one styling approach and stick to it
 - Keep styles maintainable and organized
 - Use CSS variables for theming
 - Consider accessibility (contrast, font sizes)
 - Test responsive designs
-

6. When to Use Each Method

Use CSS Modules When

- Building large applications
- Need full CSS capabilities (pseudo-classes, media queries, animations)
- Need component-scoped styles
- Want to avoid global style conflicts
- Working in a team environment

Use Inline Styles When

- Styles depend on component state or props
- Prototyping or rapid development
- Building small, self-contained components
- Need dynamic calculations
- Want to avoid CSS files completely